Network-Based Diffusion Analysis

This document provides a unified framework for performing Bayesian analysis of social transmission using Network-Based Diffusion Analysis (NBDA) models from Chimento and Hoppitt (n.d.) original work. Below, we detail the implementation of various models using the BayesForge (**BF**) package via Python, R, and Julia. All examples assume data is loaded using standard `STbayes` structures Chimento and Hoppitt (n.d.).

General Principles

The principle idea behind Network Based Diffusion analysis (NBDA) is that if social transmission is involved in the spread of a novel behavior through a group, then that spread is expected to follow a social network links Hasenjager, Leadbeater, and Hoppitt (2021). The basic model underlying NBDA states that at time t an individual, i, learns the behavior of interest with a specific rate formula.

In principle NBDA can be consider as a survival analysis, so we have the same concepts as in chapter 14:

1. Where the **baseline hazard** (e.g. the hazard when all covariates are zero) is the asocial hazard.
2. Where the **covariate** is the sum of links toward informed individuals (i.e. individuals that acquired the behavior of interest at time $t-1$).
3. Thus the **Hazard Function** which account for the network links weights **covariate** can thus be consider as the social rate of learning the behavior.

Data

The examples below read `nbda_data.json`: the example data of the STbayes package (Chimento and Hoppitt (n.d.)) prepared for **BF**, with the acquisition events of 50 individuals in 1 trial and their undirected social network. Download it from Test/3.Documentation/data, where `make_nbda_data.R` shows how it is built.

Python

```python
from BayesForge import bf
import json
import jax.numpy as jnp

m = bf(platform='cpu')

# STbayes example data (50 individuals, 1 trial, 1 network), prepared for BF in nbda_data.jso
# A (network: trials x networks x time x P x P), Z (informed individuals), D (durations),
# obs_end_time / is_event / valid_ind (time of acquisition or censoring of each individual)
with open('nbda_data.json') as f:
    data_list = {k: jnp.array(v) if isinstance(v, list) else v for k, v in json.load(f).items
data_list['Zn'] = data_list['Z']  # informed individuals seen by the naive ones
```

R

```r
library(BayesForge)
m <- importBF(platform = "cpu")
jnp <- reticulate::import("jax.numpy")

# STbayes example data (50 individuals, 1 trial, 1 network), prepared for BF in nbda_data.jso
# A (network: trials x networks x time x P x P), Z (informed individuals), D (durations),
# obs_end_time / is_event / valid_ind (time of acquisition or censoring of each individual)
raw <- jsonlite::fromJSON("nbda_data.json")
data_list <- lapply(raw, function(v) if (length(v) > 1 || is.array(v)) jnp$array(v) else v)
data_list$Zn <- data_list$Z  # informed individuals seen by the naive ones
```

Julia

```julia
using PyBayesForge
using PythonCall
m = importBF(platform="cpu")

# STbayes example data (50 individuals, 1 trial, 1 network), prepared for BF in nbda_data.jso
# A (network: trials x networks x time x P x P), Z (informed individuals), D (durations),
# obs_end_time / is_event / valid_ind (time of acquisition or censoring of each individual)
json = pyimport("json")
raw = json.load(pybuiltins.open("nbda_data.json"))
```

```
data_list = pydict(Dict(string(k) => (pyisinstance(v, pybuiltins.list) ? jnp.array(v) : v)
                        for (k, v) in raw.items()))
data_list["Zn"] = data_list["Z"]  # informed individuals seen by the naive ones
```

Continuous Time of Acquisition Data Analysis (cTADA)

General Principles

The cTADA model is used when the exact continuous times of behavioural acquisition are known. It estimates a baseline intrinsic learning rate (λ_0) and a social transmission rate (s). The hazard of learning for an individual depends on their innate capacity to learn independently and the additive influence of their informed social connections over time.

Considerations

> **Note**
>
> - **Priors**: We assign Normal priors on the log-scale for λ_0 and s' (e.g., $Normal(-4, 2)$) to ensure strictly positive rates after exponentiation.
> - **Data**: Requires a survival-type data structure detailing the exact `obs_end_time` (when the event occurred or was censored) and `is_event` status.
> - **Networks**: Computes the additive effect of knowledgeable individuals connected to the naive individual across given networks.

Example

Python

```
from BayesForge import bf
import jax.numpy as jnp

m = bf(platform='cpu')
m.data_on_model = {'data': data_list}

def model_cTADA(data):
    log_lambda_0_mean = m.dist.normal(-4, 2, name="log_lambda_0_mean")
    log_s_prime_mean = m.dist.normal(-4, 2, name="log_s_prime_mean")
```

```
    lambda_0 = jnp.exp(log_lambda_0_mean)
    s_prime = jnp.exp(log_s_prime_mean)

    # Informed connections of each individual at each time step
    A_sum = jnp.sum(data['A'], axis=0)
    Z_exp = jnp.expand_dims(data['Z'], axis=-1)
    A_Z = jnp.squeeze(jnp.matmul(A_sum, Z_exp), axis=-1)
    soc_term = s_prime * A_Z

    D_exp = jnp.expand_dims(data['D'], axis=-1)
    lambda_all = (lambda_0 + soc_term) * D_exp

    # Survival likelihood: hazard accumulated while naive, log-hazard at the acquisition time
    time_grid_exp = jnp.arange(data['T_max'])[None, :, None]          # (1, T_max, 1)
    obs_end_time_exp = jnp.expand_dims(data['obs_end_time'], axis=1)  # (K, 1, P)
    sum_mask = time_grid_exp < obs_end_time_exp
    log_mask = (time_grid_exp == (obs_end_time_exp - 1)) & jnp.expand_dims(data['is_event'],
    neg_lambda_sum = jnp.sum(-lambda_all * sum_mask, axis=1)
    log_lambda_term = jnp.sum(jnp.log(lambda_all) * log_mask, axis=1)
    total_log_lik = jnp.sum((neg_lambda_sum + log_lambda_term) * data['valid_ind'])
    m.dist.unit(log_factor=total_log_lik, name="cTADA_lik")

m.fit(model_cTADA)
m.summary()
```

R

```
m <- importBF(platform = "cpu")
m$data_on_model <- list(data = data_list)

model_cTADA <- function(data) {
  log_lambda_0_mean <- m$dist$normal(-4, 2, name = "log_lambda_0_mean")
  log_s_prime_mean <- m$dist$normal(-4, 2, name = "log_s_prime_mean")

  lambda_0 <- jnp$exp(log_lambda_0_mean)
  s_prime <- jnp$exp(log_s_prime_mean)

  # Informed connections of each individual at each time step
  A_sum <- jnp$sum(data$A, axis = 0L)
  Z_exp <- jnp$expand_dims(data$Z, axis = -1L)
```

```
  A_Z <- jnp$squeeze(jnp$matmul(A_sum, Z_exp), axis = -1L)
  soc_term <- s_prime * A_Z

  D_exp <- jnp$expand_dims(data$D, axis = -1L)
  lambda_all <- (lambda_0 + soc_term) * D_exp

  # Survival likelihood: hazard accumulated while naive, log-hazard at the acquisition time
  time_grid_exp <- jnp$expand_dims(jnp$expand_dims(jnp$arange(data$T_max), axis = 0L), axis =
  obs_end_time_exp <- jnp$expand_dims(data$obs_end_time, axis = 1L)
  sum_mask <- time_grid_exp < obs_end_time_exp
  log_mask <- (time_grid_exp == (obs_end_time_exp - 1L)) & jnp$expand_dims(data$is_event, axi
  neg_lambda_sum <- jnp$sum(-lambda_all * sum_mask, axis = 1L)
  log_lambda_term <- jnp$sum(jnp$log(lambda_all) * log_mask, axis = 1L)
  total_log_lik <- jnp$sum((neg_lambda_sum + log_lambda_term) * data$valid_ind)
  m$dist$unit(log_factor = total_log_lik, name = "cTADA_lik")
}

m$fit(model_cTADA)
m$summary()
```

Julia

```
m = importBF(platform="cpu")
m.data_on_model = pydict(data = data_list)

@BF function model_cTADA(data)
    log_lambda_0_mean = m.dist.normal(-4, 2, name="log_lambda_0_mean")
    log_s_prime_mean = m.dist.normal(-4, 2, name="log_s_prime_mean")

    lambda_0 = jnp.exp(log_lambda_0_mean)
    s_prime = jnp.exp(log_s_prime_mean)

    # Informed connections of each individual at each time step
    A_sum = jnp.sum(data["A"], axis=0)
    Z_exp = jnp.expand_dims(data["Z"], axis=-1)
    A_Z = jnp.squeeze(jnp.matmul(A_sum, Z_exp), axis=-1)
    soc_term = s_prime * A_Z

    D_exp = jnp.expand_dims(data["D"], axis=-1)
    lambda_all = (lambda_0 + soc_term) * D_exp
```

```
    # Survival likelihood: hazard accumulated while naive, log-hazard at the acquisition time
    time_grid_exp = jnp.expand_dims(jnp.expand_dims(jnp.arange(data["T_max"]), axis=0), axis=
    obs_end_time_exp = jnp.expand_dims(data["obs_end_time"], axis=1)
    sum_mask = jnp.less(time_grid_exp, obs_end_time_exp)
    log_mask = jnp.logical_and(jnp.equal(time_grid_exp, obs_end_time_exp - 1),
                               jnp.expand_dims(data["is_event"], axis=1))
    neg_lambda_sum = jnp.sum(jnp.negative(lambda_all) * sum_mask, axis=1)
    log_lambda_term = jnp.sum(jnp.log(lambda_all) * log_mask, axis=1)
    total_log_lik = jnp.sum((neg_lambda_sum + log_lambda_term) * data["valid_ind"])
    m.dist.unit(log_factor=total_log_lik, name="cTADA_lik")
end

m.fit(model_cTADA)
m.summary()
```

Mathematical Details

Frequentist Formulation

The hazard rate $\lambda_i(t)$ for individual i at time t is typically defined as:

$$\lambda_i(t) = \lambda_0(t) + s \sum_j a_{i,j} z_j(t)$$

Where: - $\lambda_0(t)$ is the baseline asocial learning rate. - s is the transmission rate per unit of connection. - $a_{i,j}$ represents the network tie strength from j to i. - $z_j(t)$ is an indicator of whether j has acquired the behavior at time t.

Bayesian Formulation

$$\log(\lambda_0) \sim \text{Normal}(\mu_0, \sigma_0)$$
$$\log(s') \sim \text{Normal}(\mu_s, \sigma_s)$$

The specific event acquisition follows a Poisson process survival likelihood evaluating negative hazard over uninformed time steps, and a positive log-hazard at the precise acquisition time cut-point.

Notes

> **Note**
>
> The cTADA model processes hazard calculations fully vectorized in JAX by computing matrices of size `(K, T_max, P)` representing individual statuses at all time cut-points, rendering it extremely computationally efficient on modern hardware.

Order of Acquisition Data Analysis (OADA)

General Principles

OADA is used when the precise times of acquisition are unknown, but the sequential *order* in which individuals acquired the behavior is known. Rather than assessing a continuous survival hazard, OADA estimates the probability that the *next* observed learner was individual i out of the pool of all remaining naive candidates.

Considerations

> **Note**
>
> - **Priors**: Only the relative social transmission rate (s') is estimated as an explicitly defined parameter, often configured with a $Normal(-4, 2)$ prior on the log scale.
> - **Asocial Variant**: The `OADA_asocial` variant restricts social transmission to 0, leaving the order predicted solely by equal intrinsic chance.
> - **Data Masking**: OADA calculates the likelihood function *only* at the sequence steps where a diffusion event is recorded, significantly trimming empty temporal observations.

Example

Python

```python
from BayesForge import bf
import jax.numpy as jnp

m = bf(platform='cpu')
m.data_on_model = {'data': data_list}
```

```python
def model_OADA(data):
    log_s_prime_mean = m.dist.normal(-4, 2, name="log_s_prime_mean")
    s_prime = jnp.exp(log_s_prime_mean)

    A_sum = jnp.sum(data['A'], axis=0)
    Z_exp = jnp.expand_dims(data['Z'], axis=-1)
    A_Z = jnp.squeeze(jnp.matmul(A_sum, Z_exp), axis=-1)
    lambda_all = 1.0 + s_prime * A_Z   # relative rate: asocial rate fixed to 1

    # Probability that the individual who learnt at each step was the one to learn
    i_lambda = jnp.sum(lambda_all * data['is_event_3d'], axis=-1)              # (K, T_max)
    Q_mask = jnp.arange(data['P']) < data['Q']
    j_rates_sum = jnp.sum(lambda_all * (1.0 - data['Z']) * Q_mask, axis=-1)   # naive indivi
    event_at_time = data['event_at_time']
    log_lik = (jnp.log(jnp.where(event_at_time, i_lambda, 1.0))
               - jnp.log(jnp.where(event_at_time, j_rates_sum, 1.0)))
    total_log_lik = jnp.sum(jnp.where(event_at_time, log_lik, 0.0))
    m.dist.unit(log_factor=total_log_lik, name="OADA_lik")

m.fit(model_OADA)
m.summary()
```

R

```r
m <- importBF(platform = "cpu")
m$data_on_model <- list(data = data_list)

model_OADA <- function(data) {
  log_s_prime_mean <- m$dist$normal(-4, 2, name = "log_s_prime_mean")
  s_prime <- jnp$exp(log_s_prime_mean)

  A_sum <- jnp$sum(data$A, axis = 0L)
  Z_exp <- jnp$expand_dims(data$Z, axis = -1L)
  A_Z <- jnp$squeeze(jnp$matmul(A_sum, Z_exp), axis = -1L)
  lambda_all <- 1.0 + s_prime * A_Z   # relative rate: asocial rate fixed to 1

  # Probability that the individual who learnt at each step was the one to learn
  i_lambda <- jnp$sum(lambda_all * data$is_event_3d, axis = -1L)             # (K, T_max)
  Q_mask <- jnp$arange(data$P) < data$Q
  j_rates_sum <- jnp$sum(lambda_all * (1.0 - data$Z) * Q_mask, axis = -1L)  # naive individua
  event_at_time <- data$event_at_time
```

```r
  log_lik <- jnp$log(jnp$where(event_at_time, i_lambda, 1.0)) -
    jnp$log(jnp$where(event_at_time, j_rates_sum, 1.0))
  total_log_lik <- jnp$sum(jnp$where(event_at_time, log_lik, 0.0))
  m$dist$unit(log_factor = total_log_lik, name = "OADA_lik")
}

m$fit(model_OADA)
m$summary()
```

Julia

```julia
m = importBF(platform="cpu")
m.data_on_model = pydict(data = data_list)

@BF function model_OADA(data)
    log_s_prime_mean = m.dist.normal(-4, 2, name="log_s_prime_mean")
    s_prime = jnp.exp(log_s_prime_mean)

    A_sum = jnp.sum(data["A"], axis=0)
    Z_exp = jnp.expand_dims(data["Z"], axis=-1)
    A_Z = jnp.squeeze(jnp.matmul(A_sum, Z_exp), axis=-1)
    lambda_all = 1.0 + s_prime * A_Z   # relative rate: asocial rate fixed to 1

    # Probability that the individual who learnt at each step was the one to learn
    i_lambda = jnp.sum(lambda_all * data["is_event_3d"], axis=-1)             # (K, T_max)
    Q_mask = jnp.less(jnp.arange(data["P"]), data["Q"])
    j_rates_sum = jnp.sum(lambda_all * (1.0 - data["Z"]) * Q_mask, axis=-1)  # naive individu
    event_at_time = data["event_at_time"]
    log_lik = jnp.log(jnp.where(event_at_time, i_lambda, 1.0)) -
              jnp.log(jnp.where(event_at_time, j_rates_sum, 1.0))
    total_log_lik = jnp.sum(jnp.where(event_at_time, log_lik, 0.0))
    m.dist.unit(log_factor=total_log_lik, name="OADA_lik")
end

m.fit(model_OADA)
m.summary()
```

Mathematical Details

Frequentist Formulation

The probability that individual i is the next to acquire the behavior is:

$$P(i|Z) = \frac{\lambda_i}{\sum_{j \notin Z} \lambda_j} = \frac{\lambda_0 + s \sum_k a_{i,k} z_k}{\sum_{j \notin Z} (\lambda_0 + s \sum_k a_{j,k} z_k)}$$

Bayesian Formulation

The hazard rate is treated relatively without needing strictly scaled intrinsic times. The fractional structure uses the total active hazard for candidate i divided by the summation of the hazard models for all currently valid elements inside the population sequence.

Notes

> **Note**
>
> Because the time is abstracted out, OADA effectively treats the base intrinsic learning rate (λ_0) as 1.0. The s' parameter is evaluated as a multiple relative to the baseline intrinsic event.

Individual-Level Variables (ILV)

General Principles

The ILV model integrates continuous, boolean, or categorical traits observed on individuals (e.g., age, sex, rank) into the baseline hazard or the social transmission capabilities. This allows testing hypotheses of whether specific traits predispose candidates to learn faster, either asocially or socially.

Considerations

> **Note**
>
> - **Priors**: The β coefficients applied to the ILVs are assigned standard normal priors, e.g., $Normal(0, 1)$, meaning a zero-centered belief about an ILV having any effect.
> - **Categorical Variables**: Transformed by matching the variable's state index and generating individual specific multipliers across combinations of learning trajectories.

Example

Python

```python
from BayesForge import bf
import jax.numpy as jnp

m = bf(platform='cpu')
# One individual-level variable (ILV) used for asocial, social and multiplicative effects
data_list['ILV_bool_ILV'] = data_list['ILV'][:, None]                                    #
data_list['ILV_cont_ILV'] = data_list['ILV']                                             #
data_list['ILV_cat_ILV'] = jnp.zeros((data_list['P'], 3)).at[:, 0].set(data_list['ILV'])  #
m.data_on_model = {'data': data_list}

def model_ILV(data):
    log_lambda_0_mean = m.dist.normal(-4, 2, name="log_lambda_0_mean")
    log_s_prime_mean = m.dist.normal(-4, 2, name="log_s_prime_mean")
    beta_ILVi_bool_ILV = m.dist.normal(0, 1, shape=(1,), name="beta_ILVi_bool_ILV")
    beta_ILVs_cont_ILV = m.dist.normal(0, 1, name="beta_ILVs_cont_ILV")
    beta_ILVm_cat_ILV = m.dist.normal(0, 1, shape=(3,), name="beta_ILVm_cat_ILV")

    lambda_0 = jnp.exp(log_lambda_0_mean)
    s_prime = jnp.exp(log_s_prime_mean)

    # ILV effects on asocial learning (i), social learning (s), and both (m)
    ind_term = jnp.exp(jnp.matmul(data['ILV_bool_ILV'], beta_ILVi_bool_ILV))
    cont_ILV_s = data['ILV_cont_ILV'] * beta_ILVs_cont_ILV
    cat_ILV_m = jnp.matmul(data['ILV_cat_ILV'], beta_ILVm_cat_ILV)

    A_sum = jnp.sum(data['A'], axis=0)
    Z_exp = jnp.expand_dims(data['Z'], axis=-1)
    A_Z = jnp.squeeze(jnp.matmul(A_sum, Z_exp), axis=-1)
    soc_term = s_prime * A_Z * jnp.exp(cont_ILV_s)

    D_exp = jnp.expand_dims(data['D'], axis=-1)
    lambda_all = jnp.exp(cat_ILV_m) * (lambda_0 * ind_term + soc_term) * D_exp

    time_grid_exp = jnp.arange(data['T_max'])[None, :, None]
    obs_end_time_exp = jnp.expand_dims(data['obs_end_time'], axis=1)
    sum_mask = time_grid_exp < obs_end_time_exp
    log_mask = (time_grid_exp == (obs_end_time_exp - 1)) & jnp.expand_dims(data['is_event'],
```

```
    neg_lambda_sum = jnp.sum(-lambda_all * sum_mask, axis=1)
    log_lambda_term = jnp.sum(jnp.log(lambda_all) * log_mask, axis=1)
    total_log_lik = jnp.sum((neg_lambda_sum + log_lambda_term) * data['valid_ind'])
    m.dist.unit(log_factor=total_log_lik, name="ILV_lik")

m.fit(model_ILV)
m.summary()
```

R

```
m <- importBF(platform = "cpu")
# One individual-level variable (ILV) used for asocial, social and multiplicative effects
P <- data_list$P
data_list$ILV_bool_ILV <- jnp$expand_dims(data_list$ILV, axis = -1L)                    # (
data_list$ILV_cont_ILV <- data_list$ILV                                                 # (
data_list$ILV_cat_ILV <- jnp$stack(list(data_list$ILV, jnp$zeros(P), jnp$zeros(P)), axis = 1L
m$data_on_model <- list(data = data_list)

model_ILV <- function(data) {
  log_lambda_0_mean <- m$dist$normal(-4, 2, name = "log_lambda_0_mean")
  log_s_prime_mean <- m$dist$normal(-4, 2, name = "log_s_prime_mean")
  beta_ILVi_bool_ILV <- m$dist$normal(0, 1, shape = reticulate::tuple(1L), name = "beta_ILVi_
  beta_ILVs_cont_ILV <- m$dist$normal(0, 1, name = "beta_ILVs_cont_ILV")
  beta_ILVm_cat_ILV <- m$dist$normal(0, 1, shape = reticulate::tuple(3L), name = "beta_ILVm_c

  lambda_0 <- jnp$exp(log_lambda_0_mean)
  s_prime <- jnp$exp(log_s_prime_mean)

  # ILV effects on asocial learning (i), social learning (s), and both (m)
  ind_term <- jnp$exp(jnp$matmul(data$ILV_bool_ILV, beta_ILVi_bool_ILV))
  cont_ILV_s <- data$ILV_cont_ILV * beta_ILVs_cont_ILV
  cat_ILV_m <- jnp$matmul(data$ILV_cat_ILV, beta_ILVm_cat_ILV)

  A_sum <- jnp$sum(data$A, axis = 0L)
  Z_exp <- jnp$expand_dims(data$Z, axis = -1L)
  A_Z <- jnp$squeeze(jnp$matmul(A_sum, Z_exp), axis = -1L)
  soc_term <- s_prime * A_Z * jnp$exp(cont_ILV_s)

  D_exp <- jnp$expand_dims(data$D, axis = -1L)
  lambda_all <- jnp$exp(cat_ILV_m) * (lambda_0 * ind_term + soc_term) * D_exp
```

```r
  time_grid_exp <- jnp$expand_dims(jnp$expand_dims(jnp$arange(data$T_max), axis = 0L), axis =
  obs_end_time_exp <- jnp$expand_dims(data$obs_end_time, axis = 1L)
  sum_mask <- time_grid_exp < obs_end_time_exp
  log_mask <- (time_grid_exp == (obs_end_time_exp - 1L)) & jnp$expand_dims(data$is_event, axi
  neg_lambda_sum <- jnp$sum(-lambda_all * sum_mask, axis = 1L)
  log_lambda_term <- jnp$sum(jnp$log(lambda_all) * log_mask, axis = 1L)
  total_log_lik <- jnp$sum((neg_lambda_sum + log_lambda_term) * data$valid_ind)
  m$dist$unit(log_factor = total_log_lik, name = "ILV_lik")
}

m$fit(model_ILV)
m$summary()
```

Julia

```julia
m = importBF(platform="cpu")
# One individual-level variable (ILV) used for asocial, social and multiplicative effects
P = data_list["P"]
data_list["ILV_bool_ILV"] = jnp.expand_dims(data_list["ILV"], axis=-1)                  # (P
data_list["ILV_cont_ILV"] = data_list["ILV"]                                            # (P
data_list["ILV_cat_ILV"] = jnp.stack(pylist([data_list["ILV"], jnp.zeros(P), jnp.zeros(P)]),
m.data_on_model = pydict(data = data_list)

@BF function model_ILV(data)
    log_lambda_0_mean = m.dist.normal(-4, 2, name="log_lambda_0_mean")
    log_s_prime_mean = m.dist.normal(-4, 2, name="log_s_prime_mean")
    beta_ILVi_bool_ILV = m.dist.normal(0, 1, shape=(1,), name="beta_ILVi_bool_ILV")
    beta_ILVs_cont_ILV = m.dist.normal(0, 1, name="beta_ILVs_cont_ILV")
    beta_ILVm_cat_ILV = m.dist.normal(0, 1, shape=(3,), name="beta_ILVm_cat_ILV")

    lambda_0 = jnp.exp(log_lambda_0_mean)
    s_prime = jnp.exp(log_s_prime_mean)

    # ILV effects on asocial learning (i), social learning (s), and both (m)
    ind_term = jnp.exp(jnp.matmul(data["ILV_bool_ILV"], beta_ILVi_bool_ILV))
    cont_ILV_s = data["ILV_cont_ILV"] * beta_ILVs_cont_ILV
    cat_ILV_m = jnp.matmul(data["ILV_cat_ILV"], beta_ILVm_cat_ILV)

    A_sum = jnp.sum(data["A"], axis=0)
    Z_exp = jnp.expand_dims(data["Z"], axis=-1)
    A_Z = jnp.squeeze(jnp.matmul(A_sum, Z_exp), axis=-1)
```

```
    soc_term = s_prime * A_Z * jnp.exp(cont_ILV_s)

    D_exp = jnp.expand_dims(data["D"], axis=-1)
    lambda_all = jnp.exp(cat_ILV_m) * (lambda_0 * ind_term + soc_term) * D_exp

    time_grid_exp = jnp.expand_dims(jnp.expand_dims(jnp.arange(data["T_max"]), axis=0), axis=
    obs_end_time_exp = jnp.expand_dims(data["obs_end_time"], axis=1)
    sum_mask = jnp.less(time_grid_exp, obs_end_time_exp)
    log_mask = jnp.logical_and(jnp.equal(time_grid_exp, obs_end_time_exp - 1),
                               jnp.expand_dims(data["is_event"], axis=1))
    neg_lambda_sum = jnp.sum(jnp.negative(lambda_all) * sum_mask, axis=1)
    log_lambda_term = jnp.sum(jnp.log(lambda_all) * log_mask, axis=1)
    total_log_lik = jnp.sum((neg_lambda_sum + log_lambda_term) * data["valid_ind"])
    m.dist.unit(log_factor=total_log_lik, name="ILV_lik")
end

m.fit(model_ILV)
m.summary()
```

Mathematical Details

Frequentist Formulation

ILVs alter the transmission framework as follows:

$$\lambda_i(t) = \exp(X_i^{\beta}) \times \left[\lambda_0 \exp(X^{\alpha}_{asocial_i}) + s \sum_j a_{i,j} z_j(t) \exp(X^{\gamma}_{social_i}) \right]$$

Where β, α, γ are vectors calculating multiplicative trait influences on global, asocial, and social transmission weights respectively.

Bayesian Formulation

$$\beta_{ILV} \sim \text{Normal}(0, 1)$$

Variables scale multiplicatively.

Notes

> **Note**
>
> The exponential conversion $\exp(V_i \times \beta)$ enforces positive hazard modifiers, ensuring trait influence scales the absolute chance of behavior generation.

Varying Effects (veff)

General Principles

Models containing varying (random) effects assign specific variance structures, typically for individual personalities or repeated behavioral trials across multiple datasets. The `veff` model offsets intrinsic (λ_0) and social (s') rates for each candidate inside the recorded sample dimensions.

Considerations

> **Note**
>
> - **Priors**: Employs hierarchical modeling. Group means $\bar{\lambda}_0$ and $\bar{s}'$ observe normal definitions, while variance σ parameters dictating the individual offsets are given Half-Normal distributions to ensure positively scaled variance matrices.
> - **Population Definition**: Relies on accurate array dimensions dynamically obtained from the JAX tracer graph (`Z$shape`) inside the model function.

Example

Python

```python
from BayesForge import bf
import jax.numpy as jnp

m = bf(platform='cpu')
m.data_on_model = {'data': data_list}

def model_veff(data):
    log_lambda_0_mean = m.dist.normal(-4, 2, name="log_lambda_0_mean")
    log_s_prime_mean = m.dist.normal(-4, 2, name="log_s_prime_mean")
```

```
    # Individual offsets (varying effects) on both rates
    P = data['Z'].shape[2]
    sigma_sprime = m.dist.half_normal(1.0, name="sigma_sprime")
    sigma_lambda0 = m.dist.half_normal(1.0, name="sigma_lambda0")
    v_sprime = m.dist.normal(0.0, sigma_sprime, shape=(P,), name="v_sprime")
    v_lambda0 = m.dist.normal(0.0, sigma_lambda0, shape=(P,), name="v_lambda0")
    s_prime = jnp.exp(log_s_prime_mean + v_sprime)      # (P,) individual social rates
    lambda_0 = jnp.exp(log_lambda_0_mean + v_lambda0)   # (P,) individual intrinsic rates

    A_sum = jnp.sum(data['A'], axis=0)
    Z_exp = jnp.expand_dims(data['Z'], axis=-1)
    A_Z = jnp.squeeze(jnp.matmul(A_sum, Z_exp), axis=-1)
    soc_term = s_prime * A_Z

    D_exp = jnp.expand_dims(data['D'], axis=-1)
    lambda_all = (lambda_0 + soc_term) * D_exp

    time_grid_exp = jnp.arange(data['T_max'])[None, :, None]
    obs_end_time_exp = jnp.expand_dims(data['obs_end_time'], axis=1)
    sum_mask = time_grid_exp < obs_end_time_exp
    log_mask = (time_grid_exp == (obs_end_time_exp - 1)) & jnp.expand_dims(data['is_event'],
    neg_lambda_sum = jnp.sum(-lambda_all * sum_mask, axis=1)
    log_lambda_term = jnp.sum(jnp.log(lambda_all) * log_mask, axis=1)
    total_log_lik = jnp.sum((neg_lambda_sum + log_lambda_term) * data['valid_ind'])
    m.dist.unit(log_factor=total_log_lik, name="veff_lik")

m.fit(model_veff)
m.summary()
```

R

```
m <- importBF(platform = "cpu")
m$data_on_model <- list(data = data_list)

model_veff <- function(data) {
  log_lambda_0_mean <- m$dist$normal(-4, 2, name = "log_lambda_0_mean")
  log_s_prime_mean <- m$dist$normal(-4, 2, name = "log_s_prime_mean")

  # Individual offsets (varying effects) on both rates
  P <- data$Z$shape[[3]]
```

```r
  sigma_sprime <- m$dist$half_normal(1.0, name = "sigma_sprime")
  sigma_lambda0 <- m$dist$half_normal(1.0, name = "sigma_lambda0")
  v_sprime <- m$dist$normal(0.0, sigma_sprime, shape = reticulate::tuple(P), name = "v_sprime
  v_lambda0 <- m$dist$normal(0.0, sigma_lambda0, shape = reticulate::tuple(P), name = "v_lamb
  s_prime <- jnp$exp(log_s_prime_mean + v_sprime)     # (P,) individual social rates
  lambda_0 <- jnp$exp(log_lambda_0_mean + v_lambda0)  # (P,) individual intrinsic rates

  A_sum <- jnp$sum(data$A, axis = 0L)
  Z_exp <- jnp$expand_dims(data$Z, axis = -1L)
  A_Z <- jnp$squeeze(jnp$matmul(A_sum, Z_exp), axis = -1L)
  soc_term <- s_prime * A_Z

  D_exp <- jnp$expand_dims(data$D, axis = -1L)
  lambda_all <- (lambda_0 + soc_term) * D_exp

  time_grid_exp <- jnp$expand_dims(jnp$expand_dims(jnp$arange(data$T_max), axis = 0L), axis =
  obs_end_time_exp <- jnp$expand_dims(data$obs_end_time, axis = 1L)
  sum_mask <- time_grid_exp < obs_end_time_exp
  log_mask <- (time_grid_exp == (obs_end_time_exp - 1L)) & jnp$expand_dims(data$is_event, axi
  neg_lambda_sum <- jnp$sum(-lambda_all * sum_mask, axis = 1L)
  log_lambda_term <- jnp$sum(jnp$log(lambda_all) * log_mask, axis = 1L)
  total_log_lik <- jnp$sum((neg_lambda_sum + log_lambda_term) * data$valid_ind)
  m$dist$unit(log_factor = total_log_lik, name = "veff_lik")
}

m$fit(model_veff)
m$summary()
```

Julia

```julia
m = importBF(platform="cpu")
m.data_on_model = pydict(data = data_list)

@BF function model_veff(data)
    log_lambda_0_mean = m.dist.normal(-4, 2, name="log_lambda_0_mean")
    log_s_prime_mean = m.dist.normal(-4, 2, name="log_s_prime_mean")

    # Individual offsets (varying effects) on both rates
    P = data["Z"].shape[2]
    sigma_sprime = m.dist.half_normal(1.0, name="sigma_sprime")
    sigma_lambda0 = m.dist.half_normal(1.0, name="sigma_lambda0")
```

```
    v_sprime = m.dist.normal(0.0, sigma_sprime, shape=(P,), name="v_sprime")
    v_lambda0 = m.dist.normal(0.0, sigma_lambda0, shape=(P,), name="v_lambda0")
    s_prime = jnp.exp(log_s_prime_mean + v_sprime)      # (P,) individual social rates
    lambda_0 = jnp.exp(log_lambda_0_mean + v_lambda0)   # (P,) individual intrinsic rates

    A_sum = jnp.sum(data["A"], axis=0)
    Z_exp = jnp.expand_dims(data["Z"], axis=-1)
    A_Z = jnp.squeeze(jnp.matmul(A_sum, Z_exp), axis=-1)
    soc_term = s_prime * A_Z

    D_exp = jnp.expand_dims(data["D"], axis=-1)
    lambda_all = (lambda_0 + soc_term) * D_exp

    time_grid_exp = jnp.expand_dims(jnp.expand_dims(jnp.arange(data["T_max"]), axis=0), axis=
    obs_end_time_exp = jnp.expand_dims(data["obs_end_time"], axis=1)
    sum_mask = jnp.less(time_grid_exp, obs_end_time_exp)
    log_mask = jnp.logical_and(jnp.equal(time_grid_exp, obs_end_time_exp - 1),
                               jnp.expand_dims(data["is_event"], axis=1))
    neg_lambda_sum = jnp.sum(jnp.negative(lambda_all) * sum_mask, axis=1)
    log_lambda_term = jnp.sum(jnp.log(lambda_all) * log_mask, axis=1)
    total_log_lik = jnp.sum((neg_lambda_sum + log_lambda_term) * data["valid_ind"])
    m.dist.unit(log_factor=total_log_lik, name="veff_lik")
end

m.fit(model_veff)
m.summary()
```

Mathematical Details

Frequentist Formulation

Usually addressed as Generalized Linear Mixed Models (GLMMs) handling "random effects":

$$\lambda_i = \lambda_0 + \nu_{0,i}$$

$v_{0,i}$ defines the unobserved error parameter assigned to offset individual i.

Bayesian Formulation

$$\sigma_{\lambda 0} \sim \text{HalfNormal}(1.0)$$

$$v_{0,i} \sim \text{Normal}(0.0, \sigma_{\lambda 0})$$

Bayesian representations natively support hierarchical estimations representing specific individuals drawing unique offsets based strictly on the overarching dataset standard deviation profile.

Notes

> **Note**
>
> This model effectively captures personality attributes when measuring overlapping traits across identical individuals, identifying distinct transmission behavior profiles.

Complex Transmission (complex_f)

General Principles

Traditional linear assumption states transmission increases uniformly with more informed connections. The `complex_f` formulation introduces an exponential modifier f governing positive or negative frequency-dependent limits. This allows modeling thresholds where behaviors only transmit after a certain *proportion* of a cohort knows the skill.

Considerations

> **Note**
>
> - **Priors**: Evaluating a network structure requires defining active (knowledgeable) connections and inactive (naive) connections. The parameter `log_f_mean` is treated under a $Normal(0, 1)$ curve.
> - **Alternative parameters**: The `complex_k` shape structure serves as a numerically stable replacement utilizing uniform sigmoid distributions instead of raw exponent variables, but achieves similar non-linear adjustments.

Example

Python

```
from BayesForge import bf
import jax.numpy as jnp

m = bf(platform='cpu')
m.data_on_model = {'data': data_list}

def model_complex_f(data):
    log_lambda_0_mean = m.dist.normal(-4, 2, name="log_lambda_0_mean")
    log_s_prime_mean = m.dist.normal(-4, 2, name="log_s_prime_mean")
    log_f_mean = m.dist.normal(0, 1, name="log_f_mean")

    lambda_0 = jnp.exp(log_lambda_0_mean)
    s_prime = jnp.exp(log_s_prime_mean)
    f = jnp.exp(log_f_mean)   # f > 1: conformist, f < 1: anti-conformist transmission

    # Frequency-dependent transmission: informed^f / (informed^f + naive^f)
    Z_exp = jnp.expand_dims(jnp.expand_dims(data['Z'], axis=-1), axis=0)
    Zn_exp = jnp.expand_dims(jnp.expand_dims(data['Zn'], axis=-1), axis=0)
    active = jnp.squeeze(jnp.matmul(data['A'], Z_exp), axis=-1)
    inactive = jnp.squeeze(jnp.matmul(data['A'], 1.0 - Zn_exp), axis=-1)
    active_f = jnp.power(active, f)
    inactive_f = jnp.power(inactive, f)
    frac = jnp.where((active + inactive) > 0, active_f / (active_f + inactive_f), 0.0)
    soc_term = s_prime * jnp.sum(frac, axis=0)

    D_exp = jnp.expand_dims(data['D'], axis=-1)
    lambda_all = (lambda_0 + soc_term) * D_exp

    time_grid_exp = jnp.arange(data['T_max'])[None, :, None]
    obs_end_time_exp = jnp.expand_dims(data['obs_end_time'], axis=1)
    sum_mask = time_grid_exp < obs_end_time_exp
    log_mask = (time_grid_exp == (obs_end_time_exp - 1)) & jnp.expand_dims(data['is_event'],
    neg_lambda_sum = jnp.sum(-lambda_all * sum_mask, axis=1)
    log_lambda_term = jnp.sum(jnp.log(lambda_all) * log_mask, axis=1)
    total_log_lik = jnp.sum((neg_lambda_sum + log_lambda_term) * data['valid_ind'])
    m.dist.unit(log_factor=total_log_lik, name="complex_f_lik")

m.fit(model_complex_f)
m.summary()
```

R

```r
m <- importBF(platform = "cpu")
m$data_on_model <- list(data = data_list)

model_complex_f <- function(data) {
  log_lambda_0_mean <- m$dist$normal(-4, 2, name = "log_lambda_0_mean")
  log_s_prime_mean <- m$dist$normal(-4, 2, name = "log_s_prime_mean")
  log_f_mean <- m$dist$normal(0, 1, name = "log_f_mean")

  lambda_0 <- jnp$exp(log_lambda_0_mean)
  s_prime <- jnp$exp(log_s_prime_mean)
  f <- jnp$exp(log_f_mean)  # f > 1: conformist, f < 1: anti-conformist transmission

  # Frequency-dependent transmission: informed^f / (informed^f + naive^f)
  Z_exp <- jnp$expand_dims(jnp$expand_dims(data$Z, axis = -1L), axis = 0L)
  Zn_exp <- jnp$expand_dims(jnp$expand_dims(data$Zn, axis = -1L), axis = 0L)
  active <- jnp$squeeze(jnp$matmul(data$A, Z_exp), axis = -1L)
  inactive <- jnp$squeeze(jnp$matmul(data$A, 1.0 - Zn_exp), axis = -1L)
  active_f <- jnp$power(active, f)
  inactive_f <- jnp$power(inactive, f)
  frac <- jnp$where((active + inactive) > 0, active_f / (active_f + inactive_f), 0.0)
  soc_term <- s_prime * jnp$sum(frac, axis = 0L)

  D_exp <- jnp$expand_dims(data$D, axis = -1L)
  lambda_all <- (lambda_0 + soc_term) * D_exp

  time_grid_exp <- jnp$expand_dims(jnp$expand_dims(jnp$arange(data$T_max), axis = 0L), axis =
  obs_end_time_exp <- jnp$expand_dims(data$obs_end_time, axis = 1L)
  sum_mask <- time_grid_exp < obs_end_time_exp
  log_mask <- (time_grid_exp == (obs_end_time_exp - 1L)) & jnp$expand_dims(data$is_event, ax
  neg_lambda_sum <- jnp$sum(-lambda_all * sum_mask, axis = 1L)
  log_lambda_term <- jnp$sum(jnp$log(lambda_all) * log_mask, axis = 1L)
  total_log_lik <- jnp$sum((neg_lambda_sum + log_lambda_term) * data$valid_ind)
  m$dist$unit(log_factor = total_log_lik, name = "complex_f_lik")
}

m$fit(model_complex_f)
m$summary()
```

Julia

```
m = importBF(platform="cpu")
m.data_on_model = pydict(data = data_list)

@BF function model_complex_f(data)
    log_lambda_0_mean = m.dist.normal(-4, 2, name="log_lambda_0_mean")
    log_s_prime_mean = m.dist.normal(-4, 2, name="log_s_prime_mean")
    log_f_mean = m.dist.normal(0, 1, name="log_f_mean")

    lambda_0 = jnp.exp(log_lambda_0_mean)
    s_prime = jnp.exp(log_s_prime_mean)
    f = jnp.exp(log_f_mean)   # f > 1: conformist, f < 1: anti-conformist transmission

    # Frequency-dependent transmission: informed^f / (informed^f + naive^f)
    Z_exp = jnp.expand_dims(jnp.expand_dims(data["Z"], axis=-1), axis=0)
    Zn_exp = jnp.expand_dims(jnp.expand_dims(data["Zn"], axis=-1), axis=0)
    active = jnp.squeeze(jnp.matmul(data["A"], Z_exp), axis=-1)
    inactive = jnp.squeeze(jnp.matmul(data["A"], 1.0 - Zn_exp), axis=-1)
    active_f = jnp.power(active, f)
    inactive_f = jnp.power(inactive, f)
    frac = jnp.where(jnp.greater(active + inactive, 0), active_f / (active_f + inactive_f), 0
    soc_term = s_prime * jnp.sum(frac, axis=0)

    D_exp = jnp.expand_dims(data["D"], axis=-1)
    lambda_all = (lambda_0 + soc_term) * D_exp

    time_grid_exp = jnp.expand_dims(jnp.expand_dims(jnp.arange(data["T_max"]), axis=0), axis=
    obs_end_time_exp = jnp.expand_dims(data["obs_end_time"], axis=1)
    sum_mask = jnp.less(time_grid_exp, obs_end_time_exp)
    log_mask = jnp.logical_and(jnp.equal(time_grid_exp, obs_end_time_exp - 1),
                               jnp.expand_dims(data["is_event"], axis=1))
    neg_lambda_sum = jnp.sum(jnp.negative(lambda_all) * sum_mask, axis=1)
    log_lambda_term = jnp.sum(jnp.log(lambda_all) * log_mask, axis=1)
    total_log_lik = jnp.sum((neg_lambda_sum + log_lambda_term) * data["valid_ind"])
    m.dist.unit(log_factor=total_log_lik, name="complex_f_lik")
end

m.fit(model_complex_f)
m.summary()
```

Mathematical Details

Frequentist Formulation

The hazard function involves fractional frequency components evaluating connected neighbors. Rather than unscaled connection summation:

$$s \times \frac{(\sum_j a_{ij} z_j)^f}{(\sum_j a_{ij} z_j)^f + (\sum_j a_{ij}(1 - z_j))^f}$$

Where f defines the transmission bias exponent. Values of $f > 1$ indicate disproportionate influence from majorities (conformity), and $f < 1$ values support extreme novelty transmission where knowledge readily spreads despite low frequency.

Bayesian Formulation

$$\log(f) \sim \text{Normal}(0, 1)$$

Notes

> **Note**
>
> Because the scale of s' shifts its domain interpretation relative to fractional connections (e.g., s becomes relative to having *all* connections highly informed rather than a single unit), comparing raw transmission weights directly between standard representations and complex curves yields improper alignment.

Dynamic Networks & Transmission Weights (dynamic_tweights)

General Principles

Some phenomena involve interactions bounded over fluctuating intervals, such as highly motile populations where connections change dynamically. Furthermore, the likelihood that individuals express specific behavioral traits (the "transmission weight") varies over time. The Dynamic networks framework implements functional transformations supporting exact temporal alignments.

Considerations

Note

- **Priors**: Employs the k-shape curve parameterization applied through local normalized bounding: mapping unconstrained standard distributions into $(-1, 1)$ domain mappings via `sigmoid` functionality.
- **Networks**: Computes interactions tracking T_{max} discrete time occurrences against the spatial coordinates.

Example

Python

```
from BayesForge import bf
import jax
import jax.numpy as jnp

m = bf(platform='cpu')
m.data_on_model = {'data': data_list}

def model_dynamic_tweights(data):
    log_lambda_0_mean = m.dist.normal(-4, 2, name="log_lambda_0_mean")
    log_s_prime_mean = m.dist.normal(-4, 2, name="log_s_prime_mean")
    k_raw = m.dist.normal(0, 3, name="k_raw")

    lambda_0 = jnp.exp(log_lambda_0_mean)
    s_prime = jnp.exp(log_s_prime_mean)
    k_shape = 2.0 * jax.nn.sigmoid(k_raw) - 1.0   # shape of the transmission curve, in (-1,

    def dini_func(x, k):
        """Dini curve: maps the proportion of informed contacts to a transmission weight."""
        x_transformed = 2.0 * x - 1.0
        return ((x_transformed - k * x_transformed)
                / (k - 2.0 * k * jnp.abs(x_transformed) + 1.0) + 1.0) / 2.0

    Z_exp = jnp.expand_dims(jnp.expand_dims(data['Z'], axis=-1), axis=0)
    Zn_exp = jnp.expand_dims(jnp.expand_dims(data['Zn'], axis=-1), axis=0)
    numer = jnp.squeeze(jnp.matmul(data['A'], Z_exp), axis=-1)
    denom = numer + jnp.squeeze(jnp.matmul(data['A'], 1.0 - Zn_exp), axis=-1)
    prop = jnp.where(denom > 0, numer / denom, 0.0)
    soc_term = s_prime * jnp.sum(dini_func(prop, k_shape), axis=0)
```

```
    D_exp = jnp.expand_dims(data['D'], axis=-1)
    lambda_all = (lambda_0 + soc_term) * D_exp

    time_grid_exp = jnp.arange(data['T_max'])[None, :, None]
    obs_end_time_exp = jnp.expand_dims(data['obs_end_time'], axis=1)
    sum_mask = time_grid_exp < obs_end_time_exp
    log_mask = (time_grid_exp == (obs_end_time_exp - 1)) & jnp.expand_dims(data['is_event'],
    neg_lambda_sum = jnp.sum(-lambda_all * sum_mask, axis=1)
    log_lambda_term = jnp.sum(jnp.log(lambda_all) * log_mask, axis=1)
    total_log_lik = jnp.sum((neg_lambda_sum + log_lambda_term) * data['valid_ind'])
    m.dist.unit(log_factor=total_log_lik, name="dynamic_tweights_lik")

m.fit(model_dynamic_tweights)
m.summary()
```

R

```
m <- importBF(platform = "cpu")
jax <- reticulate::import("jax")
m$data_on_model <- list(data = data_list)

model_dynamic_tweights <- function(data) {
  log_lambda_0_mean <- m$dist$normal(-4, 2, name = "log_lambda_0_mean")
  log_s_prime_mean <- m$dist$normal(-4, 2, name = "log_s_prime_mean")
  k_raw <- m$dist$normal(0, 3, name = "k_raw")

  lambda_0 <- jnp$exp(log_lambda_0_mean)
  s_prime <- jnp$exp(log_s_prime_mean)
  k_shape <- 2.0 * jax$nn$sigmoid(k_raw) - 1.0  # shape of the transmission curve, in (-1, 1)

  # Dini curve: maps the proportion of informed contacts to a transmission weight
  dini_func <- function(x, k) {
    x_transformed <- 2.0 * x - 1.0
    ((x_transformed - k * x_transformed) /
      (k - 2.0 * k * jnp$abs(x_transformed) + 1.0) + 1.0) / 2.0
  }

  Z_exp <- jnp$expand_dims(jnp$expand_dims(data$Z, axis = -1L), axis = 0L)
  Zn_exp <- jnp$expand_dims(jnp$expand_dims(data$Zn, axis = -1L), axis = 0L)
  numer <- jnp$squeeze(jnp$matmul(data$A, Z_exp), axis = -1L)
```

```
  denom <- numer + jnp$squeeze(jnp$matmul(data$A, 1.0 - Zn_exp), axis = -1L)
  prop <- jnp$where(denom > 0, numer / denom, 0.0)
  soc_term <- s_prime * jnp$sum(dini_func(prop, k_shape), axis = 0L)

  D_exp <- jnp$expand_dims(data$D, axis = -1L)
  lambda_all <- (lambda_0 + soc_term) * D_exp

  time_grid_exp <- jnp$expand_dims(jnp$expand_dims(jnp$arange(data$T_max), axis = 0L), axis =
  obs_end_time_exp <- jnp$expand_dims(data$obs_end_time, axis = 1L)
  sum_mask <- time_grid_exp < obs_end_time_exp
  log_mask <- (time_grid_exp == (obs_end_time_exp - 1L)) & jnp$expand_dims(data$is_event, ax
  neg_lambda_sum <- jnp$sum(-lambda_all * sum_mask, axis = 1L)
  log_lambda_term <- jnp$sum(jnp$log(lambda_all) * log_mask, axis = 1L)
  total_log_lik <- jnp$sum((neg_lambda_sum + log_lambda_term) * data$valid_ind)
  m$dist$unit(log_factor = total_log_lik, name = "dynamic_tweights_lik")
}

m$fit(model_dynamic_tweights)
m$summary()
```

Julia

```
m = importBF(platform="cpu")
m.data_on_model = pydict(data = data_list)

@BF function model_dynamic_tweights(data)
    log_lambda_0_mean = m.dist.normal(-4, 2, name="log_lambda_0_mean")
    log_s_prime_mean = m.dist.normal(-4, 2, name="log_s_prime_mean")
    k_raw = m.dist.normal(0, 3, name="k_raw")

    lambda_0 = jnp.exp(log_lambda_0_mean)
    s_prime = jnp.exp(log_s_prime_mean)
    k_shape = 2.0 * jax.nn.sigmoid(k_raw) - 1.0   # shape of the transmission curve, in (-1,

    # Dini curve: maps the proportion of informed contacts to a transmission weight
    function dini_func(x, k)
        x_transformed = 2.0 * x - 1.0
        return ((x_transformed - k * x_transformed) /
                (k - 2.0 * k * jnp.abs(x_transformed) + 1.0) + 1.0) / 2.0
    end
```

```
    Z_exp = jnp.expand_dims(jnp.expand_dims(data["Z"], axis=-1), axis=0)
    Zn_exp = jnp.expand_dims(jnp.expand_dims(data["Zn"], axis=-1), axis=0)
    numer = jnp.squeeze(jnp.matmul(data["A"], Z_exp), axis=-1)
    denom = numer + jnp.squeeze(jnp.matmul(data["A"], 1.0 - Zn_exp), axis=-1)
    prop = jnp.where(jnp.greater(denom, 0), numer / denom, 0.0)
    soc_term = s_prime * jnp.sum(dini_func(prop, k_shape), axis=0)

    D_exp = jnp.expand_dims(data["D"], axis=-1)
    lambda_all = (lambda_0 + soc_term) * D_exp

    time_grid_exp = jnp.expand_dims(jnp.expand_dims(jnp.arange(data["T_max"]), axis=0), axis=
    obs_end_time_exp = jnp.expand_dims(data["obs_end_time"], axis=1)
    sum_mask = jnp.less(time_grid_exp, obs_end_time_exp)
    log_mask = jnp.logical_and(jnp.equal(time_grid_exp, obs_end_time_exp - 1),
                               jnp.expand_dims(data["is_event"], axis=1))
    neg_lambda_sum = jnp.sum(jnp.negative(lambda_all) * sum_mask, axis=1)
    log_lambda_term = jnp.sum(jnp.log(lambda_all) * log_mask, axis=1)
    total_log_lik = jnp.sum((neg_lambda_sum + log_lambda_term) * data["valid_ind"])
    m.dist.unit(log_factor=total_log_lik, name="dynamic_tweights_lik")
end

m.fit(model_dynamic_tweights)
m.summary()
```

Mathematical Details

Frequentist Formulation

Dynamic matrices represent adjacency parameters changing per timeline configuration interval t. Thus a_{ij} becomes $a_{ij}(t)$. Similarly, transmission variables map internal traits $w_{ij}(t)$. Combined under the Dini transform fraction, frequency models trace precise granular interactions evaluated uniformly.

Bayesian Formulation

$$k_{raw} \sim \text{Normal}(0, 3)$$

Variables scale within standard sigmoid structures ensuring efficient algorithmic bounds tracking.

Notes

> **Note**
>
> Dynamic transmission parameters require "High-Resolution" data configurations where variables adjust independently across varying observation intervals rather than single global constraints.

Network Uncertainty (posterior_edges)

General Principles

Networks are commonly abstracted from noisy sampling data featuring unequal effort constraints across dyads (missing observations). Rather than employing strict binary thresholds for associations, the `posterior_edges` model implements a multivariate structure predicting internal edge connection likelihoods directly scaled from generated network models (like STRAND or BISoN).

Considerations

> **Note**
>
> - **Priors**: The network connections rely on estimating `edge_logit`, using existing mean array (`logit_edge_mu`) and covariance matrix data (`logit_edge_cov`) extracted from independent network models feeding directly into the diffusion structure.
> - **Networks**: Internally generated A association properties are built by manipulating sampled bounds mapping $P \times P$ matrices through JAX index arrays defining symmetric bidirectional edges.

Example

Python

```
from BayesForge import bf
import jax
import jax.numpy as jnp
```

```
m = bf(platform='cpu')
# Edge weights known only up to a posterior (e.g. from an edge model, see Network Edges Infe
# here a standard normal on the logit scale for each of the P(P-1)/2 dyads
P = data_list['P']
focal_ID, other_ID = jnp.triu_indices(P, k=1)
N_dyad = focal_ID.shape[0]
data_list['N_dyad'] = N_dyad
data_list['logit_edge_mu'] = jnp.zeros((1, N_dyad))
data_list['logit_edge_cov'] = jnp.eye(N_dyad)[None]
data_list['focal_ID'] = focal_ID + 1   # 1-based ids, as STbayes
data_list['other_ID'] = other_ID + 1
m.data_on_model = {'data': data_list}

def model_posterior_edges(data):
    log_lambda_0_mean = m.dist.normal(-4, 2, name="log_lambda_0_mean")
    log_s_prime_mean = m.dist.normal(-4, 2, name="log_s_prime_mean")
    lambda_0 = jnp.exp(log_lambda_0_mean)
    s_prime = jnp.exp(log_s_prime_mean)

    # Sample the edge weights from their posterior and build the network
    edge_logit = m.dist.multivariate_normal(loc=data['logit_edge_mu'],
                                            covariance_matrix=data['logit_edge_cov'],
                                            name="edge_logit")
    w = jax.nn.sigmoid(edge_logit)
    network_idx = jnp.arange(data['N_networks'])[:, None]
    focal_idx = data['focal_ID'] - 1
    other_idx = data['other_ID'] - 1
    A = jnp.zeros((data['N_networks'], data['P'], data['P']))
    A = A.at[network_idx, focal_idx, other_idx].set(w)
    A = A.at[network_idx, other_idx, focal_idx].set(w)
    A_sum = jnp.sum(A, axis=0)[None, None]   # same network at every trial and time step

    Z_exp = jnp.expand_dims(data['Z'], axis=-1)
    A_Z = jnp.squeeze(jnp.matmul(A_sum, Z_exp), axis=-1)
    soc_term = s_prime * A_Z

    D_exp = jnp.expand_dims(data['D'], axis=-1)
    lambda_all = (lambda_0 + soc_term) * D_exp

    time_grid_exp = jnp.arange(data['T_max'])[None, :, None]
    obs_end_time_exp = jnp.expand_dims(data['obs_end_time'], axis=1)
    sum_mask = time_grid_exp < obs_end_time_exp
```

```
    log_mask = (time_grid_exp == (obs_end_time_exp - 1)) & jnp.expand_dims(data['is_event'],
    neg_lambda_sum = jnp.sum(-lambda_all * sum_mask, axis=1)
    log_lambda_term = jnp.sum(jnp.log(lambda_all) * log_mask, axis=1)
    total_log_lik = jnp.sum((neg_lambda_sum + log_lambda_term) * data['valid_ind'])
    m.dist.unit(log_factor=total_log_lik, name="posterior_edges_lik")

m.fit(model_posterior_edges)
m.summary()
```

R

```
m <- importBF(platform = "cpu")
jax <- reticulate::import("jax")
# Edge weights known only up to a posterior (e.g. from an edge model, see Network Edges Infe
# here a standard normal on the logit scale for each of the P(P-1)/2 dyads
P <- data_list$P
dyads <- jnp$triu_indices(P, k = 1L)
N_dyad <- dyads[[1]]$shape[[1]]
data_list$N_dyad <- N_dyad
data_list$logit_edge_mu <- jnp$zeros(reticulate::tuple(1L, N_dyad))
data_list$logit_edge_cov <- jnp$expand_dims(jnp$eye(N_dyad), axis = 0L)
data_list$focal_ID <- dyads[[1]] + 1L  # 1-based ids, as STbayes
data_list$other_ID <- dyads[[2]] + 1L
m$data_on_model <- list(data = data_list)

model_posterior_edges <- function(data) {
  log_lambda_0_mean <- m$dist$normal(-4, 2, name = "log_lambda_0_mean")
  log_s_prime_mean <- m$dist$normal(-4, 2, name = "log_s_prime_mean")
  lambda_0 <- jnp$exp(log_lambda_0_mean)
  s_prime <- jnp$exp(log_s_prime_mean)

  # Sample the edge weights from their posterior and build the network
  edge_logit <- m$dist$multivariate_normal(loc = data$logit_edge_mu,
                                           covariance_matrix = data$logit_edge_cov,
                                           name = "edge_logit")
  w <- jax$nn$sigmoid(edge_logit)
  network_idx <- jnp$expand_dims(jnp$arange(data$N_networks), axis = -1L)
  focal_idx <- data$focal_ID - 1L
  other_idx <- data$other_ID - 1L
  A <- jnp$zeros(reticulate::tuple(data$N_networks, data$P, data$P))
  A <- A$at[network_idx, focal_idx, other_idx]$set(w)
```

```
  A <- A$at[network_idx, other_idx, focal_idx]$set(w)
  # same network at every trial and time step
  A_sum <- jnp$expand_dims(jnp$expand_dims(jnp$sum(A, axis = 0L), axis = 0L), axis = 0L)

  Z_exp <- jnp$expand_dims(data$Z, axis = -1L)
  A_Z <- jnp$squeeze(jnp$matmul(A_sum, Z_exp), axis = -1L)
  soc_term <- s_prime * A_Z

  D_exp <- jnp$expand_dims(data$D, axis = -1L)
  lambda_all <- (lambda_0 + soc_term) * D_exp

  time_grid_exp <- jnp$expand_dims(jnp$expand_dims(jnp$arange(data$T_max), axis = 0L), axis =
  obs_end_time_exp <- jnp$expand_dims(data$obs_end_time, axis = 1L)
  sum_mask <- time_grid_exp < obs_end_time_exp
  log_mask <- (time_grid_exp == (obs_end_time_exp - 1L)) & jnp$expand_dims(data$is_event, axi
  neg_lambda_sum <- jnp$sum(-lambda_all * sum_mask, axis = 1L)
  log_lambda_term <- jnp$sum(jnp$log(lambda_all) * log_mask, axis = 1L)
  total_log_lik <- jnp$sum((neg_lambda_sum + log_lambda_term) * data$valid_ind)
  m$dist$unit(log_factor = total_log_lik, name = "posterior_edges_lik")
}

m$fit(model_posterior_edges)
m$summary()
```

Julia

```
m = importBF(platform="cpu")
# Edge weights known only up to a posterior (e.g. from an edge model, see Network Edges Infe
# here a standard normal on the logit scale for each of the P(P-1)/2 dyads
P = data_list["P"]
dyads = jnp.triu_indices(P, k=1)
N_dyad = dyads[0].shape[0]
data_list["N_dyad"] = N_dyad
data_list["logit_edge_mu"] = jnp.zeros((1, N_dyad))
data_list["logit_edge_cov"] = jnp.expand_dims(jnp.eye(N_dyad), axis=0)
data_list["focal_ID"] = dyads[0] + 1   # 1-based ids, as STbayes
data_list["other_ID"] = dyads[1] + 1
m.data_on_model = pydict(data = data_list)

@BF function model_posterior_edges(data)
    log_lambda_0_mean = m.dist.normal(-4, 2, name="log_lambda_0_mean")
```

```
    log_s_prime_mean = m.dist.normal(-4, 2, name="log_s_prime_mean")
    lambda_0 = jnp.exp(log_lambda_0_mean)
    s_prime = jnp.exp(log_s_prime_mean)

    # Sample the edge weights from their posterior and build the network
    edge_logit = m.dist.multivariate_normal(loc=data["logit_edge_mu"],
                                            covariance_matrix=data["logit_edge_cov"],
                                            name="edge_logit")
    w = jax.nn.sigmoid(edge_logit)
    network_idx = jnp.expand_dims(jnp.arange(data["N_networks"]), axis=-1)
    focal_idx = data["focal_ID"] - 1
    other_idx = data["other_ID"] - 1
    A = jnp.zeros((data["N_networks"], data["P"], data["P"]))
    A = A.at[network_idx, focal_idx, other_idx].set(w)
    A = A.at[network_idx, other_idx, focal_idx].set(w)
    # same network at every trial and time step
    A_sum = jnp.expand_dims(jnp.expand_dims(jnp.sum(A, axis=0), axis=0), axis=0)

    Z_exp = jnp.expand_dims(data["Z"], axis=-1)
    A_Z = jnp.squeeze(jnp.matmul(A_sum, Z_exp), axis=-1)
    soc_term = s_prime * A_Z

    D_exp = jnp.expand_dims(data["D"], axis=-1)
    lambda_all = (lambda_0 + soc_term) * D_exp

    time_grid_exp = jnp.expand_dims(jnp.expand_dims(jnp.arange(data["T_max"]), axis=0), axis=
    obs_end_time_exp = jnp.expand_dims(data["obs_end_time"], axis=1)
    sum_mask = jnp.less(time_grid_exp, obs_end_time_exp)
    log_mask = jnp.logical_and(jnp.equal(time_grid_exp, obs_end_time_exp - 1),
                               jnp.expand_dims(data["is_event"], axis=1))
    neg_lambda_sum = jnp.sum(jnp.negative(lambda_all) * sum_mask, axis=1)
    log_lambda_term = jnp.sum(jnp.log(lambda_all) * log_mask, axis=1)
    total_log_lik = jnp.sum((neg_lambda_sum + log_lambda_term) * data["valid_ind"])
    m.dist.unit(log_factor=total_log_lik, name="posterior_edges_lik")
end

m.fit(model_posterior_edges)
m.summary()
```

Mathematical Details

Frequentist Formulation

Standard applications represent associations identically tracking uniform interactions across specific point coordinates (a_{ij}). Values lacking sample observations resolve missing attributes as strictly void constraints reducing baseline interaction.

Bayesian Formulation

$$a_{ij} \sim \text{MultivariateNormal}(\mu_{dyad}, \Sigma_{dyad})$$

Integrating network boundaries resolves the joint uncertainty structure allowing accurate estimates mapping population distributions rather than raw sample limits, dramatically eliminating false negative assertions representing untethered population elements.

Notes

> **Note**
>
> This model prevents the severe over-estimation of intrinsic (λ_0) properties observed frequently executing standardized unweighted networks tracking under-sampled elements.

Model validation tests

- Comparison with STbayes: NBDA models fitted with BayesForge and STbayes, posterior densities and timing compared.
- Documentation examples: the Python, R and Julia examples of this page are run by the documentation tests (`Python/test_model_pages.py`, `R/test_r_model_pages.py`, `Julia/test_julia_model_pages.py`).

Chimento, Michael, and William Hoppitt. n.d. "STbayes: An r Package for Creating, Fitting and Understanding Bayesian Models of Social Transmission." *Methods in Ecology and Evolution* n/a (n/a). https://doi.org/https://doi.org/10.1111/2041-210x.70228.

Hasenjager, Matthew J., Ellouise Leadbeater, and William Hoppitt. 2021. "Detecting and Quantifying Social Transmission Using Network-Based Diffusion Analysis." *Journal of Animal Ecology* 90 (1): 8–26. https://doi.org/https://doi.org/10.1111/1365-2656.13307.